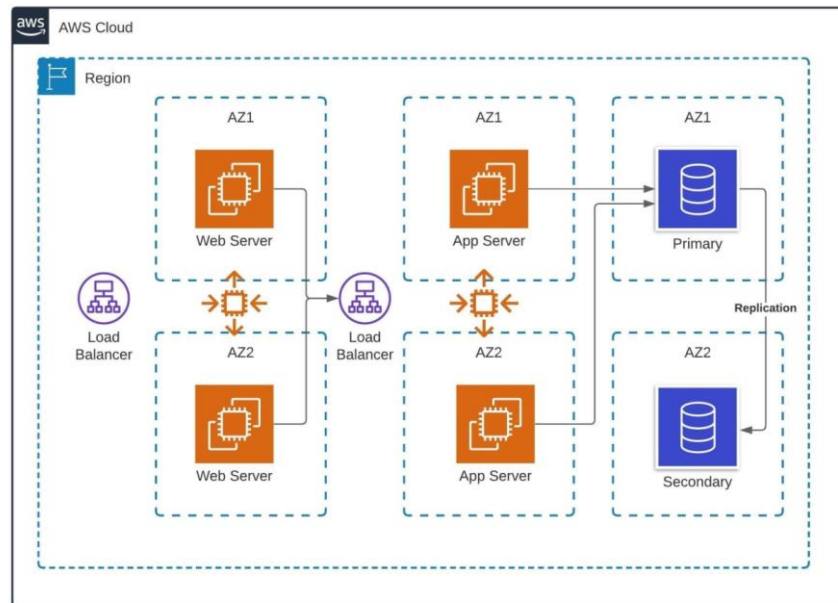




Université Amadou Mahtar MBOW de Dakar
Ecole Supérieure Polytech Diamniadio
Ingénierie des Systèmes d'Information et Données



AWS 3-Tier Web Architecture

Déploiement d'une architecture Web 3-tier sur AWS

Présenté par **M. Maguette DIOP**

Sous la supervision de **Dr. Massamba LO**

Support de référence

AWS Three Tier Web Application, K21Academy, édition 10.

08/08/2026

Table des matières

1. Introduction	4
1.1 Qu'est-ce qu'une architecture 3-tier ?	4
1.2 Objectifs	4
2. Architecture cible et vue d'ensemble	5
2.1 Services AWS utilisés	5
2.2 Pourquoi ces choix ?	6
3. Préparation des ressources transverses	7
3.1 Bucket Amazon S3 — dépôt du code	7
3.2 Rôle IAM pour EC2 — accès S3 et Session Manager	7
4. Construction du réseau	9
4.1 VPC et sous-réseaux	9
4.2 Connectivité Internet	10
4.2.1 Internet Gateway	10
4.2.2 NAT Gateways	11
4.3 Tables de routage	11
4.4 Security Groups	12
5. Déploiement de la couche de données (Amazon Aurora)	13
5.1 DB Subnet Group	13
5.2 Cluster Aurora MySQL	13
6. Déploiement de la couche applicative (App Tier)	15
6.1 Lancement de l'instance	15
6.2 Connexion et configuration	15
6.3 Test local	15
7. Déploiement de l'App Tier (ALB interne + Auto Scaling)	16
7.1 AMI de l'App Tier	16
7.2 Target Group App	16
7.3 Application Load Balancer interne	16
7.4 Launch Template App	17
7.5 Auto Scaling Group App	17
8. Déploiement de la couche de présentation (Web Tier)	18

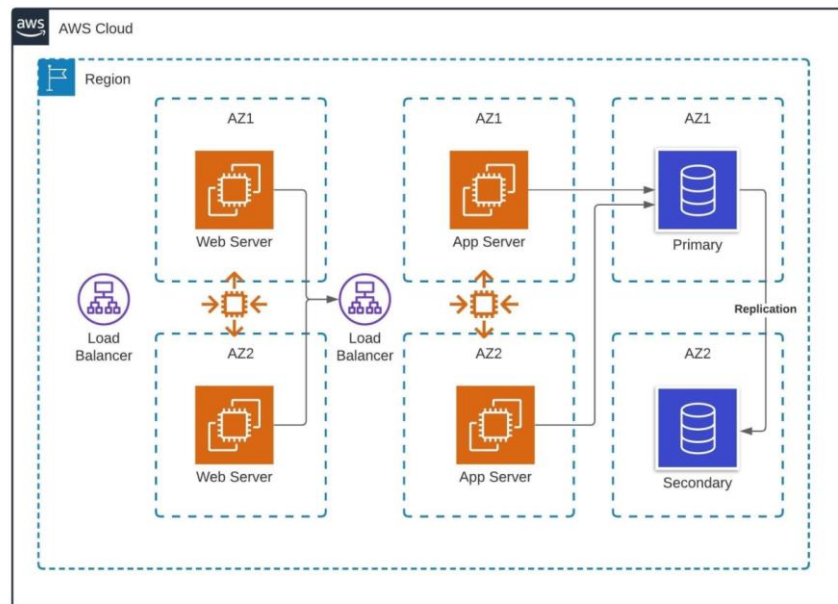
8.1 Mise à jour de nginx.conf	18
8.2 Lancement et configuration de l'instance Web	18
9. Deploiement du Web Tier (ALB Internet-facing + Auto Scaling).....	19
9.1 AMI du Web Tier.....	19
9.2 Target Group Web.....	19
9.3 Application Load Balancer Internet-facing	19
9.4 Launch Template et Auto Scaling Group Web	19
10. Vérification de bout en bout	21
11. Bonnes pratiques mobilisées et retour critique.....	22
11.1 Bonnes pratiques appliquées.....	22
11.2 Limites de cette implémentation.....	22
11.3 Pistes d'amélioration.....	22
12. Nettoyage des ressources	24
13. Conclusion	25
14. Références	26

1. Introduction

1.1 Qu'est-ce qu'une architecture 3-tier ?

Une architecture **3-tier** organise une application en trois couches distinctes. La **couche de présentation** (*Web Tier*) affiche l'interface et envoie les pages HTML, CSS et JavaScript au navigateur. La **couche logique** (*App Tier*) contient le code métier : elle traite les demandes, applique les règles et interroge la base. La **couche de données** (*Data Tier*) stocke les informations et garantit leur intégrité.

Ce découpage a trois avantages. Il rend la maintenance plus simple, car chaque couche a un rôle clair. Il permet ensuite de faire évoluer chaque couche indépendamment selon la charge. Il améliore enfin la sécurité, puisque seule la couche de présentation est exposée à Internet ; l'application et les données restent isolées.



1.2 Objectifs

Ce TP reprend le scénario d'une entreprise fictive, *Company X*, organise un événement annuel qui attire beaucoup de trafic et souhaite héberger son site sur AWS. Le site doit être scalable, disponible en permanence, sécurisé, et reposer sur des services managés.

Le TP vise à mettre en pratique cinq compétences : construire un VPC réparti sur plusieurs zones de disponibilité, appliquer le principe du moindre privilège avec IAM et les *Security Groups*, déployer une base de données managée (Aurora) en multi-AZ, automatiser le déploiement d'instances EC2 via *AMI*, *Launch Templates*, *Auto Scaling Groups* et *Application Load Balancers*, et enfin vérifier le bon fonctionnement d'une vraie application web (React + Node.js + MySQL).

2. Architecture cible et vue d'ensemble

L'architecture est déployée sur deux zones de disponibilité, *us-east-1a* et *us-east-1b*, pour assurer la haute disponibilité. Chaque zone contient trois sous-réseaux : un public pour la couche Web, un privé pour la couche App et un privé pour la couche Data.

Deux **Application Load Balancers** encadrent la couche applicative. Le premier, *Internet-facing*, reçoit le trafic des utilisateurs et le distribue aux instances Web. Le second, *interne*, sépare la couche Web de la couche App et sert de point d'entrée à l'API. Deux **Auto Scaling Groups** maintiennent deux instances par couche, réparties sur les deux AZ. La couche de données repose sur un cluster **Aurora MySQL** avec un nœud d'écriture et un nœud de lecture placés dans des AZ différentes.

2.1 Services AWS utilisés

Service	Couche	Rôle dans l'architecture
Amazon S3	Transverse	Stocke le code source, lu par les instances via IAM
AWS IAM	Transverse	Rôle EC2 donnant accès à S3 et à Systems Manager
Amazon VPC	Réseau	Réseau privé virtuel isolé
Subnets	Réseau	Découpage public/privé par AZ et par couche
Internet Gateway	Réseau	Passerelle entrée/sortie pour la couche Web
NAT Gateway	Réseau	Sortie Internet contrôlée pour les couches privées
Route Tables	Réseau	Aiguillage du trafic vers IGW ou NAT
Security Groups	Sécurité	Pare-feux au niveau de chaque instance
Amazon RDS Aurora	Données	Cluster MySQL managé, multi-AZ
Amazon EC2	Calcul	Instances Web (Nginx + React) et App (Node.js + PM2)
AMI	Calcul	Image d'instance figée après configuration
Launch Template	Calcul	Modèle de lancement (AMI, rôle IAM, SG, etc.)
Auto Scaling Group	Calcul	Maintient le nombre d'instances voulues, réparties multi-AZ
Application Load Balancer	Trafic	Répartit les requêtes, vérifie la santé, sépare les couches
Target Group	Trafic	Groupe de cibles derrière un ALB, avec protocole et port
AWS Systems Manager	Exploitation	Connexion sécurisée aux instances (Session Manager) sans clé SSH

2.2 Pourquoi ces choix ?

Deux AZ, c'est le minimum demandé par AWS pour un ALB ou un cluster Aurora en haute disponibilité : si une AZ tombe, l'autre prend le relais sans coupure. La séparation Web/App/Data suit une logique en cascade : la base n'est accessible que depuis l'App, l'App n'est accessible que depuis l'ALB interne, et l'ALB interne n'est accessible que depuis la couche Web. La surface exposée à Internet se limite donc à la seule couche Web.

L'ALB interne joue un rôle clé : il sépare la couche Web, qui bouge souvent et qui est visible, de la couche App, plus stable et plus sensible. Une évolution d'un côté ne casse pas l'autre. Enfin, le trio **AMI + Launch Template + Auto Scaling Group** apporte deux garanties : l'immuabilité (une instance en panne n'est pas réparée, elle est remplacée par une copie identique) et l'élasticité (le nombre d'instances suit la charge).

3. Préparation des ressources transverses

Avant de construire le réseau, deux ressources partagées sont mises en place : un **bucket S3** qui héberge le code source, et un **rôle IAM** que les instances EC2 utiliseront pour accéder à S3 et à Systems Manager.

3.1 Bucket Amazon S3 — dépôt du code

Un bucket S3 avec un nom unique est créé dans la région us-east-1, avec le blocage total de l'accès public. S3 est un stockage objet très durable (11 « 9 » de durabilité), séparé du cycle de vie des instances EC2. Il permet de préparer le code une fois pour toutes et de le fournir à toute instance qui en a besoin, sans copie manuelle. Bloquer l'accès public suit la recommandation *AWS S3 Block Public Access* : les objets ne sont jamais visibles depuis Internet, seule une identité authentifiée peut les lire.

Une fois le bucket créé, les dossiers `app-tier/` (API Node.js) et `web-tier/` (application React et fichier Nginx) issus du dépôt `aws-samples/aws-three-tier-web-architecture-workshop` y sont déposés. Ils serviront de source unique aux futures instances.

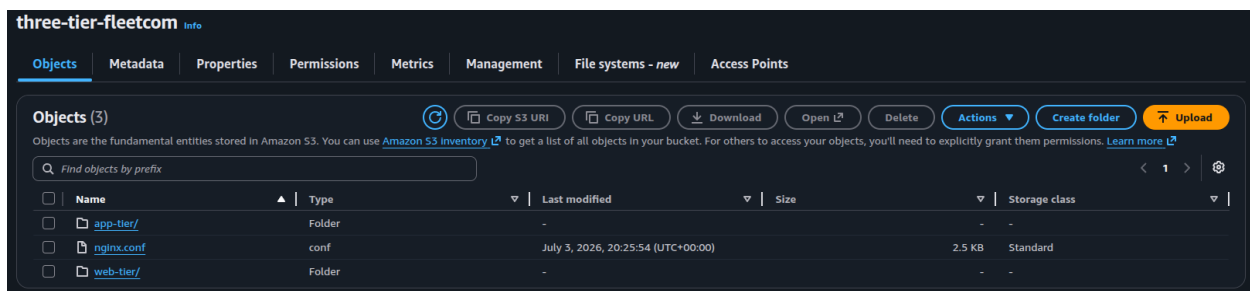


Figure 1 — Bucket S3 hébergeant le code source (dossiers `app-tier/` et `web-tier/`).

3.2 Rôle IAM pour EC2 — accès S3 et Session Manager

Un rôle IAM de type *AWS service* → *EC2* est créé avec deux politiques managées. La première, `AmazonS3ReadOnlyAccess`, permet aux instances de télécharger le code déposé à l'étape précédente sans qu'il soit nécessaire de stocker une clé d'accès en clair — ce qui évite une grande partie des fuites de secrets. La seconde, `AmazonSSMManagedInstanceCore`, permet à l'instance de s'enregistrer auprès d'AWS Systems Manager.

Grâce à cette seconde politique, la connexion administrateur passe par **Session Manager**, qui ouvre un shell à travers l'API AWS. Il n'y a plus besoin d'ouvrir le port SSH (22), plus besoin de gérer des clés, et chaque session est enregistrée dans CloudTrail. C'est la posture recommandée par AWS pour les environnements de production.

three-tier-IAM-role [info](#) [Delete](#)

Allows EC2 instances to call AWS services on your behalf.

Summary [Edit](#)

Creation date July 03, 2026, 17:50 (UTC)	ARN arn:aws:iam::162214592853:role/three-tier-IAM-role	Instance profile ARN arn:aws:iam::162214592853:instance-profile/three-tier-IAM-role
Last activity -	Maximum session duration 1 hour	

Permissions | Trust relationships | Tags | Last Accessed | Revoke sessions

Permissions policies (2) [info](#) [Simulate](#) [Remove](#) [Add permissions](#)

You can attach up to 10 managed policies.

Search Filter by Type [All types](#) [1](#) [Settings](#)

☐	Policy name ↗	Type	Attached entities
☐	AmazonS3ReadOnlyAccess	AWS managed	1
☐	AmazonSSMManagedInstanceCore	AWS managed	1

Figure 2 — Rôle IAM utilisable par EC2, avec les permissions S3 (lecture) et Systems Manager.

4. Construction du réseau

Les ressources partagées étant prêtes, l'étape suivante est le réseau privé virtuel. C'est lui qui définit la topologie et les frontières de sécurité dans lesquelles seront placés la base de données puis les instances.

4.1 VPC et sous-réseaux

Un VPC dédié, nommé `threetierapp-vpc`, est créé avec le bloc CIDR `10.0.0.0/16`. Six sous-réseaux en /24 y sont provisionnés, répartis symétriquement sur les deux AZ :

Nom	AZ	CIDR	Rôle
Public-Web-Subnet-AZ-1	us-east-1a	10.0.0.0/24	Web (public)
Public-Web-Subnet-AZ-2	us-east-1b	10.0.1.0/24	Web (public)
Private-App-Subnet-AZ-1	us-east-1a	10.0.2.0/24	App (privé)
Private-App-Subnet-AZ-2	us-east-1b	10.0.3.0/24	App (privé)
Private-DB-Subnet-AZ-1	us-east-1a	10.0.4.0/24	Data (privé)
Private-DB-Subnet-AZ-2	us-east-1b	10.0.5.0/24	Data (privé)

Le VPC forme une enveloppe isolée : rien ne peut y entrer sans autorisation. Le découpage en sous-réseaux permet d'appliquer un routage et une sécurité différents selon la couche et la zone. Le nom `<Visibilité>-<Couche>-Subnet-AZ-N` rend la topologie facile à lire et à auditer.

Les captures ci-dessous montrent la vue d'ensemble du VPC et la liste des six sous-réseaux.

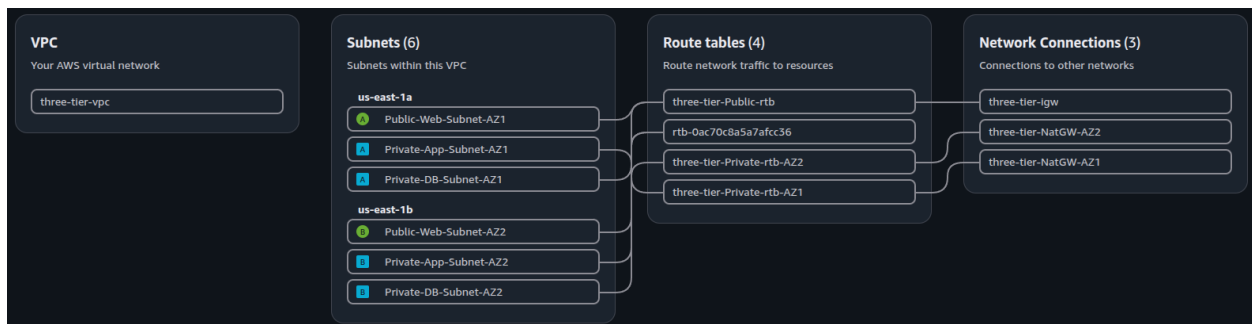


Figure 3 — Cartographie du VPC: deux AZ, trois couches, six sous-réseaux, deux tables de routage privées et une table publique.

Name	Subnet ID	State	VPC	Block Public...	IPv4 CIDR	IPv6 CIDR	IPv6 CIDR
Private-App-Subnet-AZ2	subnet-07bd6603625161735	Available	vpc-0d44fe49e98ea3282 thre...	Off	10.0.3.0/24	-	-
Private-DB-Subnet-AZ2	subnet-0f2137b1239fd3909	Available	vpc-0d44fe49e98ea3282 thre...	Off	10.0.5.0/24	-	-
Private-DB-Subnet-AZ1	subnet-015427c3699c6dbae4	Available	vpc-0d44fe49e98ea3282 thre...	Off	10.0.4.0/24	-	-
Public-Web-Subnet-AZ2	subnet-0c84614c3eadb9207	Available	vpc-0d44fe49e98ea3282 thre...	Off	10.0.1.0/24	-	-
Public-Web-Subnet-AZ1	subnet-064f19b06adcd1d3d9	Available	vpc-0d44fe49e98ea3282 thre...	Off	10.0.0.0/24	-	-
Private-App-Subnet-AZ1	subnet-07150cb0e79aeb05b	Available	vpc-0d44fe49e98ea3282 thre...	Off	10.0.2.0/24	-	-

Figure 4 — Sous-réseaux du VPC (état « Available », rattachés à `threetierapp-vpc`).

Les zooms sur chaque couche d'une même AZ confirment le placement attendu :

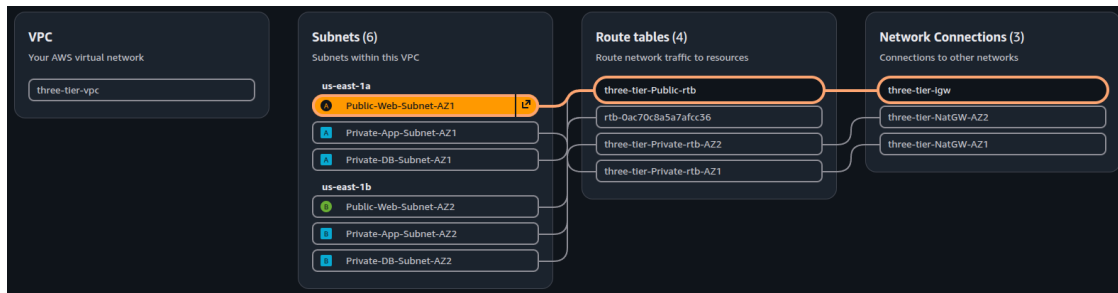


Figure 5 — Sous-réseau public *Public-Web-Subnet-AZ-1* (10.0.0.0/24).

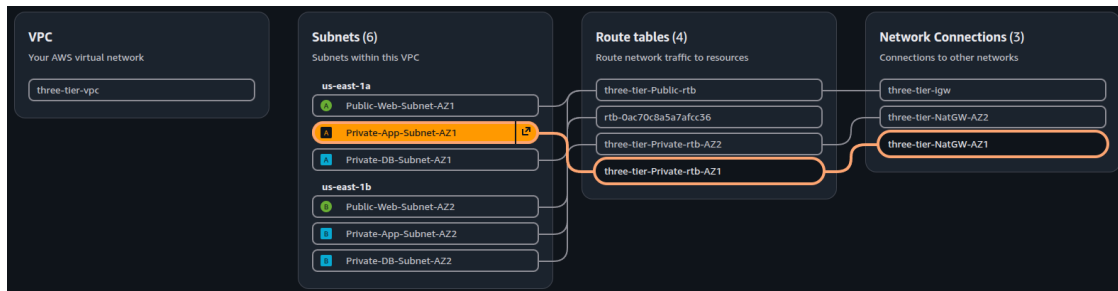


Figure 6 — Sous-réseau privé *Private-App-Subnet-AZ-1* (10.0.2.0/24).

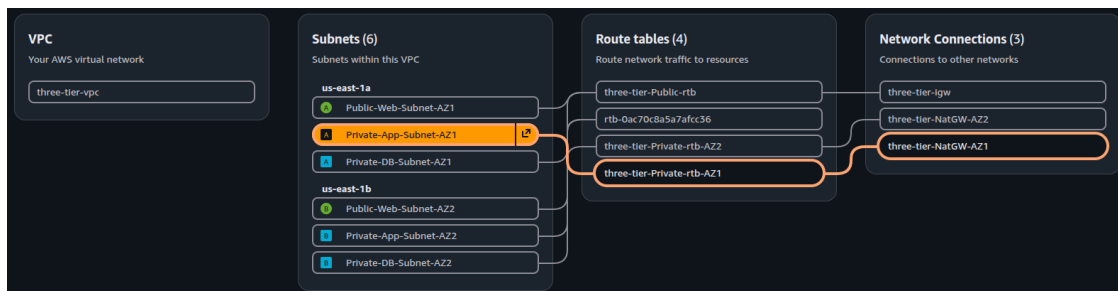


Figure 7 — Sous-réseau privé *Private-DB-Subnet-AZ-1* (10.0.4.0/24).

4.2 Connectivité Internet

Deux besoins de connectivité sont à couvrir. La couche Web publique doit pouvoir échanger avec Internet dans les deux sens. La couche App privée n'a besoin que d'une sortie (par exemple pour installer des paquets) sans pouvoir être jointe depuis l'extérieur.

4.2.1 Internet Gateway

Une passerelle Internet nommée *three-tier-igw* est créée puis attachée au VPC. C'est le seul composant qui permet à une IP publique du VPC de communiquer avec Internet dans les deux sens. Elle est entièrement managée, hautement disponible et scalable : elle ne devient jamais un goulot d'étranglement.

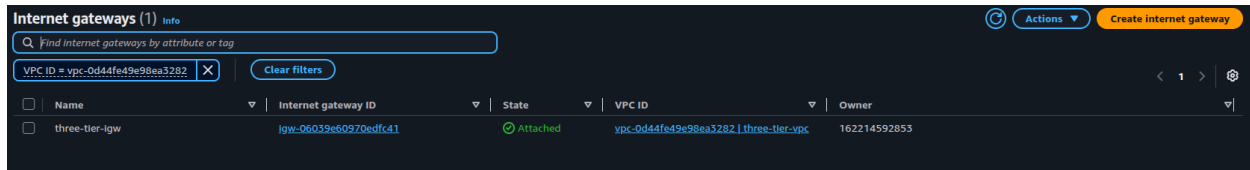


Figure 8 — Internet Gateway three-tier-igw en état « Attached » sur threetierapp-vpc.

4.2.2 NAT Gateways

Deux passerelles NAT, NAT-GW-AZ1 et NAT-GW-AZ2, sont ensuite créées, une dans chaque sous-réseau public, avec une Elastic IP dédiée. Le fait d'avoir une NAT par AZ n'est pas un détail : même si la NAT Gateway est un service régional, son plan de données vit dans une seule AZ. En avoir une par zone garantit qu'un incident sur une AZ n'empêche pas les instances privées de l'autre AZ de sortir sur Internet. Avec une seule NAT, la panne de son AZ paralyserait la moitié saine de l'infrastructure.

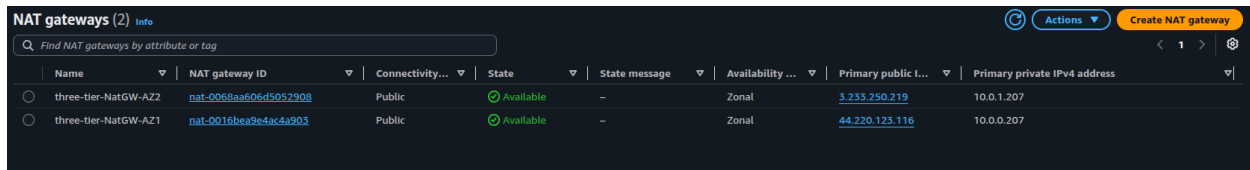


Figure 9 — NAT Gateways déployées, une par AZ, chacune avec une Elastic IP.

4.3 Tables de routage

Trois tables de routage définissent l'aiguillage du trafic. `PublicRouteTable`, associée aux deux sous-réseaux publics Web, ajoute une route par défaut `0.0.0.0/0` vers `three-tier-igw`. `PrivateRouteTable-AZ1`, associée aux sous-réseaux privés App et DB de l'AZ 1, envoie le même trafic vers NAT-GW-AZ1. `PrivateRouteTable-AZ2` fait de même dans l'AZ 2 en pointant vers NAT-GW-AZ2.

Un sous-réseau ne devient « public » ou « privé » qu'à travers sa table de routage. Séparer les tables privées par AZ concrétise le choix d'avoir une NAT par AZ : chaque sous-réseau privé sort par la NAT de sa propre zone, ce qui évite un aller-retour entre AZ (coûteux) et supprime le point unique de panne.

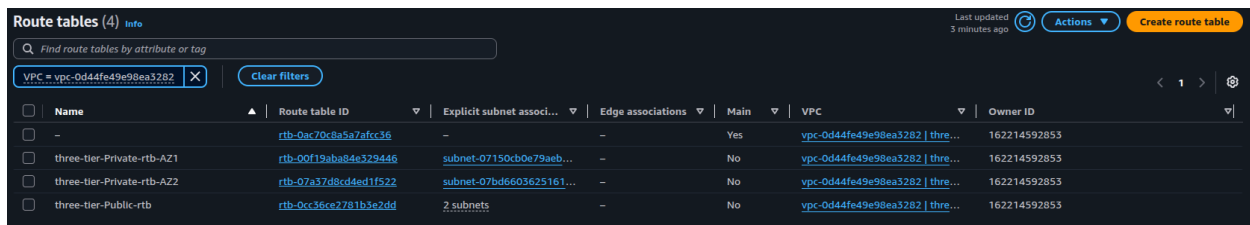


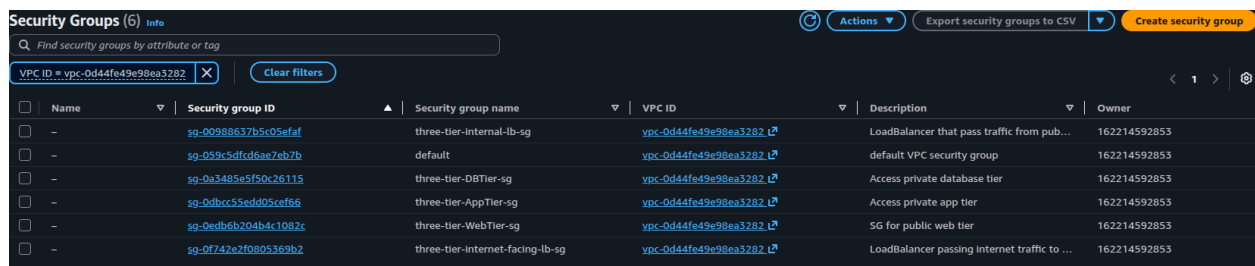
Figure 10 — Tables de routage du VPC, avec leurs associations de sous-réseaux.

4.4 Security Groups

Cinq *Security Groups* sont créés, formant une chaîne d'accès qui descend couche par couche :

SG	Trafic entrant autorisé	Rôle
internet-facing-lb-sg	HTTP (80) depuis 0.0.0.0/0	ALB externe: accepte le trafic utilisateur
WebTier-sg	HTTP (80) depuis internet-facing-lb-sg et depuis <i>My IP</i>	Instances Web: reçoivent l'ALB externe
internal-lb-sg	HTTP (80) depuis WebTier-sg	ALB interne : reçoit le trafic Web
Privateinstance-sg	TCP/4000 depuis internal-lb-sg et depuis <i>My IP</i>	Instances App : reçoivent l'ALB interne
DB-sg	MySQL/Aurora (3306) depuis Privateinstance-sg	Cluster Aurora : accessible seulement depuis l'App

Indiquer un autre Security Group comme source (plutôt qu'une plage CIDR) est une bonne pratique importante : la règle suit automatiquement les instances qui rejoignent ou quittent ce SG, sans avoir à mettre à jour d'adresses IP. La chaîne obtenue applique le moindre privilège réseau : la base de données n'accepte jamais une connexion directe depuis Internet ni depuis un serveur Web ; seul un serveur App du bon SG peut l'atteindre.



Name	Security group ID	Security group name	VPC ID	Description	Owner
-	sg-00988637b5c05efaf	three-tier-internal-lb-sg	vpc-0d44fe49e98ea3282	LoadBalancer that pass traffic from pub...	162214592853
-	sg-059c5dfcd6ae7eb7b	default	vpc-0d44fe49e98ea3282	default VPC security group	162214592853
-	sg-0a3485e5f50c26115	three-tier-DBTier-sg	vpc-0d44fe49e98ea3282	Access private database tier	162214592853
-	sg-0dbcc5edd05cef66	three-tier-AppTier-sg	vpc-0d44fe49e98ea3282	Access private app tier	162214592853
-	sg-0ed6b6b204b4c1082c	three-tier-WebTier-sg	vpc-0d44fe49e98ea3282	SG for public web tier	162214592853
-	sg-0f742e7f0805369b2	three-tier-internet-facing-lb-sg	vpc-0d44fe49e98ea3282	LoadBalancer passing internet traffic to ...	162214592853

Figure 11 — Chaîne de Security Groups définissant qui peut parler à qui entre les couches.

Les règles supplémentaires sur *My IP* pour WebTier-sg (HTTP 80) et Privateinstance-sg (TCP 4000) sont temporaires : elles servent aux tests manuels avant la mise en place des ALB et seront retirées en production.

5. Déploiement de la couche de données (Amazon Aurora)

Le réseau et la sécurité étant en place, la couche Data est déployée en premier : les instances applicatives auront besoin, dès leur démarrage, de pouvoir se connecter à la base pour créer le schéma.

5.1 DB Subnet Group

Un *DB Subnet Group* nommé `three-tier-db-subnet-group` regroupe les deux sous-réseaux `Private-DB-Subnet-AZ-1` (`10.0.4.0/24`) et `Private-DB-Subnet-AZ-2` (`10.0.5.0/24`). RDS et Aurora ont besoin de savoir exactement dans quels sous-réseaux ils peuvent placer les nœuds du cluster. C'est ce subnet group qui permet à Aurora de répartir automatiquement un nœud d'écriture et un nœud de lecture dans deux AZ différentes.

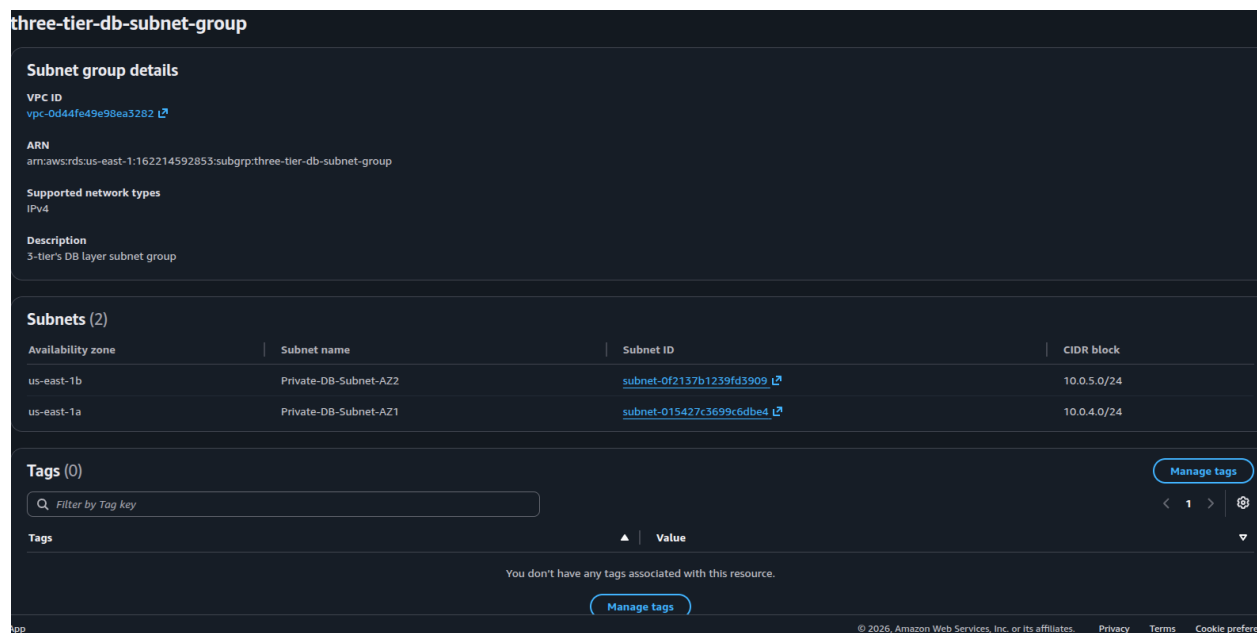


Figure 12 — DB Subnet Group `three-tier-db-subnet-group`.

5.2 Cluster Aurora MySQL

Le cluster est créé avec les paramètres suivants : moteur **Aurora (MySQL Compatible)**, modèle **Dev/Test**, instance **db.r5.large**, option « *Create an Aurora Replica in a different AZ* » activée, accès public désactivé, Security Group DB-sg, utilisateur maître admin.

Aurora est préféré à un MySQL classique car c'est un moteur relationnel managé conçu par AWS. Il reste compatible MySQL au niveau protocolaire, mais son stockage est distribué et répliqué six fois sur trois AZ. Le basculement du nœud d'écriture prend en général moins de trente secondes et se fait sans perte de données ; les sauvegardes sont continues et incrémentales. Ce sont exactement les propriétés attendues pour une application confrontée au pic de trafic décrit dans le scénario Company X.

Activer la réplique multi-AZ ajoute un nœud lecteur dans l'autre AZ. Ce nœud sert de cible de basculement automatique si la primaire tombe, et il peut en parallèle prendre en charge une partie du trafic de lecture, ce qui améliore la scalabilité en SELECT.

DB Identifier	Status	Role	Engine	Upgrade rollout order	Region ...	Size	Recommendations
database-1	Available	Regional cluster	Aurora MySQL	SECOND	us-east-1	2 instances	2 Informational
database-1-instance-1	Available	Writer instance	Aurora MySQL	SECOND	us-east-1b	Aurora serverless (2...	
database-1-instance-1-reader	Available	Reader instance	Aurora MySQL	SECOND	us-east-1a	Aurora serverless (2...	

Figure 13 — Cluster Aurora MySQL : nœud d'écriture (Writer) et nœud de lecture (Reader) dans deux AZ.

Le *endpoint* du writer, du type `<cluster>-instance-1.<hash>.us-east-1.rds.amazonaws.com`, est noté ; il servira de valeur pour `DB_HOST` dans le fichier de configuration de la couche App (app-tier/DbConfig.js).

6. Déploiement de la couche applicative (App Tier)

À ce stade, le réseau est prêt et la base répond. La couche applicative est déployée d'abord à la main sur une seule instance, pour vérifier que tout fonctionne. Elle sera ensuite figée en AMI puis automatisée derrière un ALB interne et un ASG à la section 7.

6.1 Lancement de l'instance

Une instance EC2 Amazon Linux 2 est lancée dans Private-App-Subnet-AZ-1, sans IP publique, avec le rôle IAM défini en section 3.2 et le Security Group Privateinstance-sg. L'absence d'IP publique confirme le caractère privé de la couche App : la sortie Internet nécessaire aux yum install passe par la NAT Gateway de l'AZ 1, et l'accès administrateur se fait par Session Manager grâce au rôle IAM.

6.2 Connexion et configuration

Depuis la console EC2, l'action *Connect* → *Session Manager* ouvre un shell directement dans l'instance. Le compte est basculé sur ec2-user, puis les paquets nécessaires sont installés : client MySQL, Node.js (via NVM) et le gestionnaire de processus pm2.

La connexion à Aurora est testée avec `mysql -h <endpoint> -u admin -p`, puis le schéma applicatif est créé :

```
CREATE DATABASE webappdb;
USE webappdb;
CREATE TABLE IF NOT EXISTS transactions (
  id INT NOT NULL AUTO_INCREMENT,
  amount DECIMAL(10,2),
  description VARCHAR(100),
  PRIMARY KEY (id)
);
```

Le fichier app-tier/DbConfig.js du dépôt est ensuite modifié pour renseigner DB_HOST, DB_USER, DB_PWD et DB_DATABASE. Le dossier app-tier/ est téléversé sur S3 puis récupéré sur l'instance avec `aws s3 cp s3://<bucket>/app-tier/ . --recursive`. Les dépendances Node sont installées (npm install) et l'API est lancée en arrière-plan avec `pm2 start index.js`. Le redémarrage automatique au reboot est activé par `pm2 startup` puis `pm2 save`.

PM2 est choisi parce qu'il surveille le processus Node.js, le relance en cas de crash et centralise les journaux, sans qu'il faille écrire un service systemd. Cela facilite aussi la génération d'une AMI prête à l'emploi.

6.3 Test local

`curl http://localhost:4000/health` renvoie la chaîne "This is the health check", et `curl http://localhost:4000/transaction` retourne une liste vide. Ces deux appels valident à la fois le processus Node.js et la connexion à Aurora.

7. Déploiement de l'App Tier (ALB interne + Auto Scaling)

L'instance App marche, mais elle est unique : elle représente un point de panne et n'offre aucune scalabilité. L'étape suivante consiste à la transformer en un modèle reproductible, puis à en lancer plusieurs copies réparties sur les deux AZ derrière un load balancer interne.

7.1 AMI de l'App Tier

Une AMI est créée à partir de l'instance App configurée, via **Actions** → **Image and templates** → **Create image**. Elle capture tout le disque : OS à jour, Node.js, PM2 et code applicatif. Toute nouvelle instance issue de cette AMI démarre déjà prête à l'emploi, sans exécuter de script de configuration. C'est le principe de l'*immutable infrastructure* : une instance en production n'est jamais modifiée sur place, elle est remplacée par une instance construite à partir d'une AMI plus récente.

Amazon Machine Images (AMIs) (2) info

Owned by me

Find AMI by attribute or tag

Recycle Bin

EC2 Image Builder

Actions

Launch instance from AMI

☐	Name	AMI name	AMI ID	Source	Owner	Visibility	Status	Creation date
☐		three-tier-WebTier-image	ami-02a9412403213bd0c	162214592853/three-tier-WebTier-image	162214592853	Private	Available	2026/07/03 20:57 GMT+0
☐		three-tier-AppTier-image	ami-0221855637e34c192	162214592853/three-tier-AppTier-image	162214592853	Private	Available	2026/07/03 19:35 GMT+0

Figure 14 — AMIs personnalisées app-tier-ami et web-tier-ami.

7.2 Target Group App

Un *Target Group* de type *instances* est ensuite défini, en HTTP sur le port 4000, dans le VPC `threetierapp-vpc`, avec `/health` comme chemin de sonde. Ce groupe sert d'intermédiaire entre l'ALB et les instances : ce sont ses sondes qui décident quelles cibles reçoivent réellement du trafic. Le chemin `/health` renvoie 200 tant que le processus Node.js répond, ce qui suffit à considérer l'instance comme saine.

7.3 Application Load Balancer interne

Un ALB en mode **internal** est déployé dans les deux sous-réseaux `Private-App-Subnet-AZ-*`, avec le SG `internal-lb-sg` et un listener HTTP:80 qui envoie les requêtes vers le Target Group App. Le mot « interne » est important : le DNS de cet ALB n'est pas résolvable depuis Internet, il n'est joignable qu'à l'intérieur du VPC. Cela protège la couche App tout en apportant les avantages d'un load balancer : répartition, sondes, terminaison de connexion, et possibilité d'ajouter plus tard un WAF ou du WebSocket.

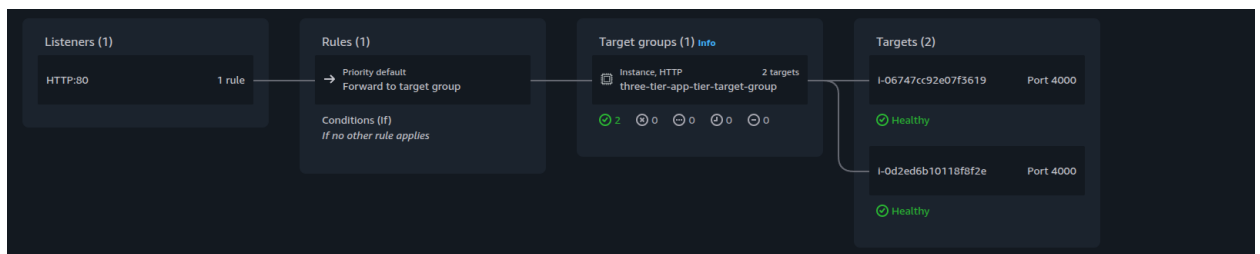


Figure 15 — Nœuds réseau de l'ALB interne, un par AZ (`Private-App-Subnet-AZ-1` et `-AZ-2`).

Le DNS de l'ALB interne, du type `internal-app-alb-
<hash>.us-east-1.elb.amazonaws.com`, est noté ; il sera utilisé dans la configuration Nginx de la couche Web.

7.4 Launch Template App

Un *Launch Template* app-tier-lt regroupe tous les paramètres de lancement : l'AMI de la section 7.1, le type d'instance t2.micro, le rôle IAM, le SG Privateinstance-sg et les sous-réseaux App privés. Ce modèle est versionné. L'Auto Scaling Group s'y référera : mettre à jour l'AMI reviendra à créer une nouvelle version du template, et l'ASG pourra effectuer un *rolling update* automatique.

7.5 Auto Scaling Group App

Un ASG app-tier-asg est lancé à partir du Launch Template, réparti sur les deux sous-réseaux App privés, avec une capacité fixée à deux instances (minimum, désiré et maximum tous à 2), et attaché au Target Group App. L'ASG remplit deux rôles : il maintient la capacité cible — si une instance tombe, il en lance automatiquement une nouvelle — et il gère l'inscription des instances dans le Target Group. La configuration à deux instances sur deux AZ assure la haute disponibilité de la couche App.

8. Déploiement de la couche de présentation (Web Tier)

Le backend est désormais accessible via le DNS de l'ALB interne. La couche Web, composée de Nginx qui sert l'application React et qui fait office de *reverse-proxy* vers l'API, est déployée de la même manière : instance de référence, test, puis mise en prod.

8.1 Mise à jour de `nginx.conf`

Dans le dossier `web-tier/` du dépôt, `nginx.conf` est modifié pour deux choses. D'abord, il sert les fichiers statiques du *build* React depuis `/home/ec2-user/web-tier/build`. Ensuite, il redirige les requêtes qui commencent par `/api/` vers le DNS de l'ALB interne avec la directive `proxy_pass http://<internal-alb-dns>:80/`. Ce montage donne au navigateur l'illusion d'une seule origine — évitant les problèmes de CORS — et permet à Nginx de traverser `internal-lb-sg` en tant que membre de `WebTier-sg`. Une fois modifiés, le dossier `web-tier/` et le fichier `nginx.conf` sont redéposés sur S3.

8.2 Lancement et configuration de l'instance Web

Une instance EC2 est lancée dans `Public-Web-Subnet-AZ-1`, avec IP publique auto-assignée, rôle IAM et Security Group `WebTier-sg`. Après connexion par Session Manager, Node.js (pour le *build* React) et Nginx sont installés, le dossier `web-tier/` est récupéré depuis S3, les commandes `npm install` puis `npm run build` sont exécutées, le dossier `build/` produit est copié vers l'emplacement servi par Nginx, le `nginx.conf` par défaut est remplacé par la version personnalisée déposée sur S3, et Nginx est démarré avec `sudo systemctl enable --now nginx`.

L'accès direct à l'IP publique de l'instance affiche la page d'accueil de l'application, ce qui valide la chaîne complète `Web → ALB interne → App → Aurora`.

9. Déploiement du Web Tier (ALB Internet-facing + Auto Scaling)

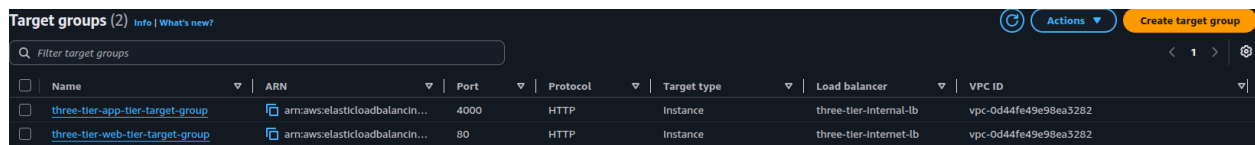
La démarche est la même qu'à la section 7 : AMI, Target Group, ALB (cette fois *Internet-facing*), Launch Template et Auto Scaling Group.

9.1 AMI du Web Tier

Une AMI web-tier-ami est créée à partir de l'instance Web configurée. Elle apparaît sur la Figure 14 à côté de l'AMI de l'App Tier.

9.2 Target Group Web

Un Target Group est défini en HTTP sur le port 80, chemin de *health check /*, type de cibles *instances*, dans le VPC threetierapp-vpc. Il servira de destination au listener de l'ALB externe.



Name	ARN	Port	Protocol	Target type	Load balancer	VPC ID
three-tier-app-tier-target-group	arn:aws:elasticloadbalancing...	4000	HTTP	Instance	three-tier-internal-lb	vpc-0d44fe49e98ea3282
three-tier-web-tier-target-group	arn:aws:elasticloadbalancing...	80	HTTP	Instance	three-tier-internet-lb	vpc-0d44fe49e98ea3282

Figure 16 — Target Groups pour l'App Tier (port 4000) et le Web Tier (port 80).

9.3 Application Load Balancer Internet-facing

Un ALB en mode **internet-facing** est déployé dans les deux sous-réseaux publics, avec le SG internet-facing-lb-sg et un listener HTTP:80 qui redirige vers le Target Group Web. Cet ALB est le seul point d'entrée public de l'architecture ; son DNS, du type `<web-alb>-<hash>.us-east-1.elb.amazonaws.com`, sera celui donné aux utilisateurs. La terminaison des connexions, le routage L7 et la répartition entre AZ sont pris en charge par AWS.

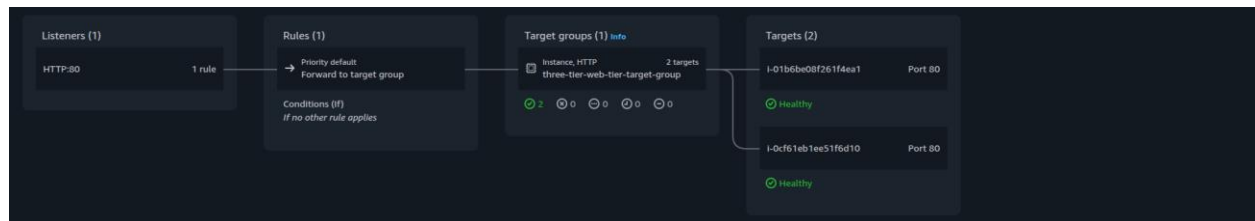
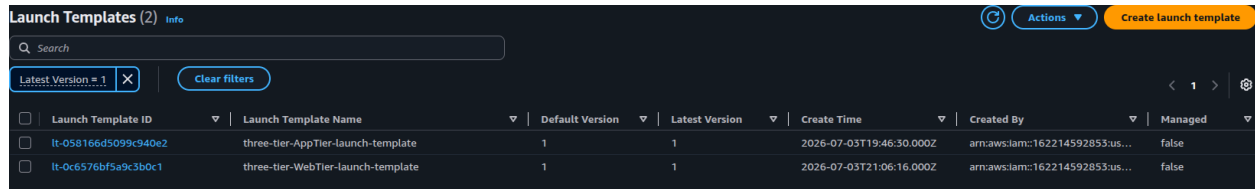


Figure 17 — Nœuds réseau de l'ALB Internet-facing, un par sous-réseau public.

9.4 Launch Template et Auto Scaling Group Web

Un Launch Template web-tier-lt référence l'AMI Web, le rôle IAM et le SG WebTier-sg. Il alimente un ASG web-tier-asg configuré à deux instances (min = desired = max = 2), attaché au Target Group Web et réparti sur les deux sous-réseaux publics. Cette étape termine la construction : les deux ALB, les deux ASG et le cluster Aurora forment ensemble une architecture multi-AZ et scriptable.



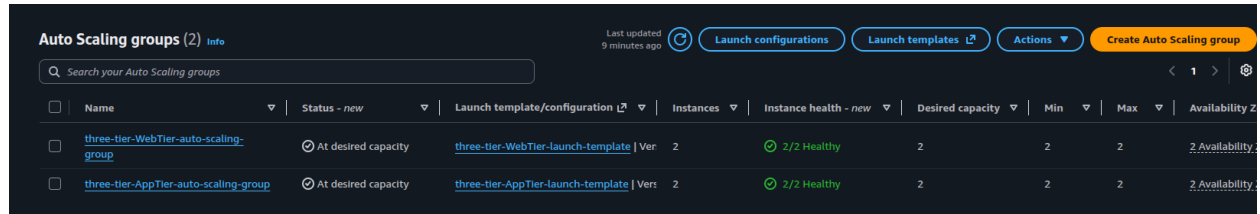
Launch Templates (2) Info

Search

Latest Version = 1 Clear filters

Launch Template ID	Launch Template Name	Default Version	Latest Version	Create Time	Created By	Managed
lt-058166d5099c940e2	three-tier-AppTier-launch-template	1	1	2026-07-03T19:46:30.000Z	arn:aws:iam::162214592853:us...	false
lt-0c6576bf5a9c3b0c1	three-tier-WebTier-launch-template	1	1	2026-07-03T21:06:16.000Z	arn:aws:iam::162214592853:us...	false

Figure 18 — Launch Templates versionnés app-tier-lt et web-tier-lt.



Auto Scaling groups (2) Info

Last updated 9 minutes ago Launch configurations Launch templates Actions Create Auto Scaling group

Search your Auto Scaling groups

Name	Status - new	Launch template/configuration	Instances	Instance health - new	Desired capacity	Min	Max	Availability Z
three-tier-WebTier-auto-scaling-group	At desired capacity	three-tier-WebTier-launch-template Ver. 2	2	2/2 Healthy	2	2	2	2 Availability Z
three-tier-AppTier-auto-scaling-group	At desired capacity	three-tier-AppTier-launch-template Ver. 2	2	2/2 Healthy	2	2	2	2 Availability Z

Figure 19 — Auto Scaling Groups maintenant deux instances par couche, réparties sur les deux AZ.

10. Vérification de bout en bout

Ouvrir dans un navigateur le DNS de l'ALB Internet-facing affiche la page d'accueil de l'application (« Three Tier Web App »).

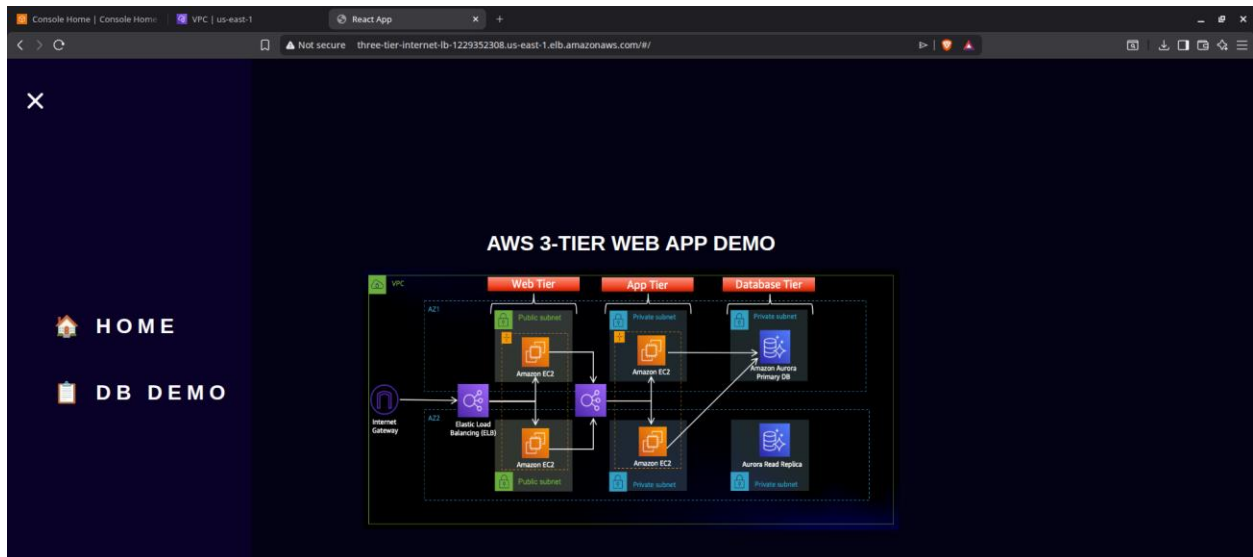


Figure 20 — Page d'accueil. La requête HTTP a traversé : navigateur → ALB externe → EC2 Web (Nginx) → ALB interne → EC2 App (Node.js) → Aurora.

La page *DB Demo* déclenche des appels POST `/api/transaction` (ajout) et GET `/api/transaction` (lecture) ; les enregistrements ajoutés sont bien stockés dans la table transactions d'Aurora puis renvoyés à la page.

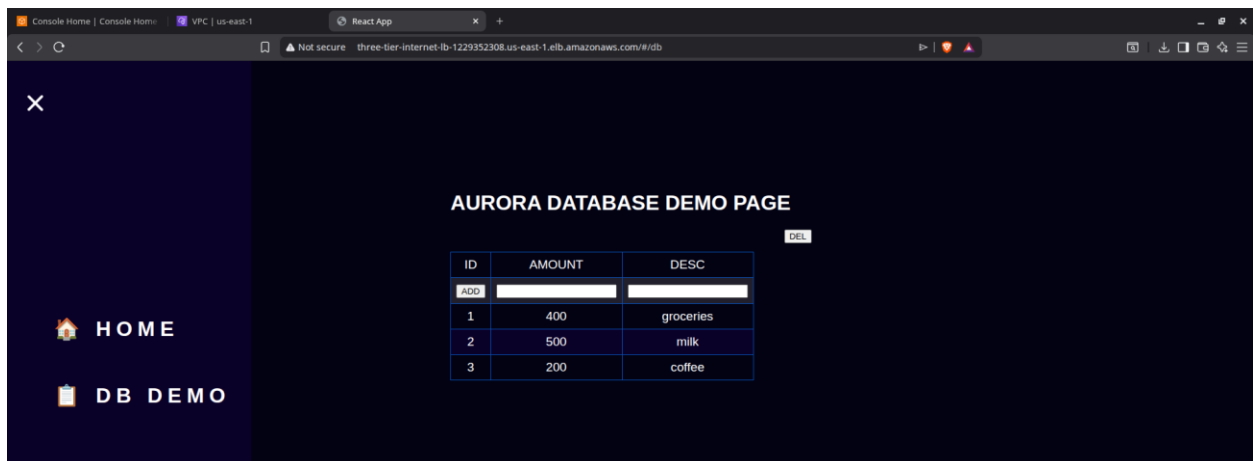


Figure 21 — Page « DB Demo » : lecture et écriture dans Aurora validées à travers toute la chaîne 3-tier.

Ce test confirme que les trois couches communiquent correctement et que les règles de sécurité sont respectées : aucune connexion directe n'est possible depuis l'extérieur vers les couches App ou Data.

11. Bonnes pratiques mobilisées et retour critique

11.1 Bonnes pratiques appliquées

L'architecture obtenue applique plusieurs bonnes pratiques recommandées par AWS. La séparation réseau public/privé par sous-réseaux et tables de routage isole clairement les surfaces exposées. Les Security Groups chaînés par référencement mutuel forment une défense en profondeur, où chaque couche n'accepte que la couche juste au-dessus. Le principe du moindre privilège IAM est respecté grâce à un rôle EC2 limité à la lecture S3 et au strict minimum Systems Manager, avec la suppression des clés SSH au profit de Session Manager. Le déploiement est multi-AZ pour Web, App, Aurora, NAT Gateways et nœuds ALB. La séparation par ALB permet aux couches Web et App d'évoluer indépendamment. Enfin, l'immuabilité est garantie par les AMI figées, chaque instance étant une copie exacte d'une image jamais modifiée en production, tandis que les ASG remplacent automatiquement les instances défaillantes.

11.2 Limites de cette implémentation

L'implémentation présente plusieurs limites qu'un vrai déploiement en production devrait corriger. Aucune terminaison HTTPS/TLS n'est en place, faute de certificat ACM et de listener HTTPS. Aucun AWS WAF n'est placé devant l'ALB externe pour filtrer les attaques applicatives. Le chiffrement au repos d'Aurora et de l'AMI n'a pas été configuré explicitement, seules les valeurs par défaut sont utilisées. Les ASG sont dimensionnés en capacité fixe (deux instances), ce qui assure la haute disponibilité mais pas l'élasticité dynamique face à la charge. Aucune alarme CloudWatch ne surveille les métriques applicatives ni les coûts. Enfin, les identifiants Aurora sont stockés en clair dans DbConfig.js alors qu'ils devraient être délégués à AWS Secrets Manager.

11.3 Pistes d'amélioration

Deux pistes se dégagent naturellement de ce retour critique.

La première est l'adoption d'une approche **Infrastructure as Code**. Toute la construction a ici été faite par clics dans la console : c'est pédagogique mais pas reproductible, pas révisable, et impossible à démanteler automatiquement. Deux outils sont pertinents. **AWS CloudFormation**, service natif AWS, s'intègre à IAM et CloudTrail et gère des *stacks* avec rollback automatique en cas d'échec. **Terraform**, édité par HashiCorp, est multi-cloud, dispose d'un très large écosystème de modules et propose la commande `terraform plan` qui affiche la différence entre l'état voulu et l'état actuel avant toute modification. Dans les deux cas, le code d'infrastructure est versionné, appliqué en CI/CD et supprimé par une seule commande (`cloudformation delete-stack` ou `terraform destroy`), ce qui remplace la plus grande partie du nettoyage manuel.

La seconde piste est la **conteneurisation** et le passage à une orchestration managée. Déployer une application *stateless* via AMI, Launch Template et ASG reste lourd : chaque mise à jour applicative impose de reconstruire une AMI, de mettre à jour le template et de lancer une *instance refresh*. Une évolution naturelle serait de conteneuriser les couches Web et App, de publier les images sur **Amazon ECR**, puis de les faire tourner sur **Amazon ECS** en mode **AWS Fargate**. Les gains sont nombreux : démarrage plus rapide qu'une instance EC2, facturation à la tâche (vCPU/heure et Go/heure) sans capacité inutilisée, intégration native avec ALB (via un target

type ip), CloudWatch Logs et IAM (grâce aux *task roles* qui donnent à chaque tâche ses propres permissions), et auto-scaling piloté par CPU, RAM ou par la métrique ALB RequestCountPerTarget, qui remplace élégamment l'ASG EC2 par un *ECS Service*. La combinaison Terraform + ECS Fargate donnerait, pour la même application, une architecture plus élastique, plus économique quand la charge est faible, et beaucoup plus rapide à faire évoluer.

12. Nettoyage des ressources

Les ressources AWS sont facturées à la durée : un nettoyage soigneux est indispensable en fin de TP. L'ordre de suppression est l'inverse de celui de la construction, en tenant compte des dépendances. Les Auto Scaling Groups (Web puis App) sont supprimés en premier, sinon les instances qu'ils supervisent seraient automatiquement recréées. Viennent ensuite les Application Load Balancers, puis les Target Groups associés, les Launch Templates, et les AMIs. Les instances EC2 encore présentes sont alors supprimées.

Le cluster Aurora est ensuite supprimé après désactivation de la *Delete protection* : on retire d'abord le reader, puis le writer, en refusant le snapshot final si aucune donnée n'est à conserver. Les NAT Gateways sont supprimées, puis les Elastic IPs sont libérées. Les tables de routage personnalisées sont supprimées avant que l'Internet Gateway ne soit détachée du VPC puis supprimée. Enfin, la suppression du VPC entraîne en cascade celle des sous-réseaux, des Security Groups et du DB Subnet Group restants.

13. Conclusion

Ce TP a permis de construire, étape par étape, une architecture web 3-tier de référence sur AWS. En partant des ressources partagées (S3, IAM), puis du réseau (VPC, sous-réseaux, IGW, NAT, tables de routage, Security Groups), en déployant ensuite la couche de données (cluster Aurora multi-AZ), la couche applicative et enfin la couche de présentation — chacune automatisée par une AMI, un Launch Template, un Auto Scaling Group et un Application Load Balancer — le TP a montré comment le catalogue AWS répond concrètement aux exigences de scalabilité, de haute disponibilité et de sécurité par isolation.

Au-delà des manipulations, l'intérêt pédagogique de ce TP est de comprendre le rôle de chaque service et les raisons des choix de conception : pourquoi une NAT par AZ, pourquoi référencer un Security Group par un autre plutôt que par une plage d'adresses, pourquoi mettre un ALB interne entre Web et App, pourquoi préférer une AMI à un script d'initialisation. On retrouve ces mêmes choix, à peu de choses près, dans toute architecture cloud sérieuse. La maquette réalisée constitue donc un socle d'apprentissage réutilisable pour explorer, plus tard, des variantes plus modernes reposant sur l'Infrastructure as Code (Cloud Formation / Terraform), les conteneurs (Amazon ECS) ou le serverless (AWS Lambda).

14. Références

- K21Academy, *AWS Three Tier Web Application*, édition 10, 2023 — AWS+3-Tier+Web+Architecture_ed10.pdf.
- Dépôt Github : <https://github.com/aws-samples/aws-three-tier-web-architecture-workshop>.
- Documentation officielle AWS :
 - Amazon VPC : <https://docs.aws.amazon.com/vpc/>
 - Amazon EC2 : <https://aws.amazon.com/ec2/>
 - Amazon RDS Aurora : <https://aws.amazon.com/rds/aurora/>
 - Amazon S3 : <https://aws.amazon.com/s3/>
 - AWS IAM Roles : https://docs.aws.amazon.com/IAM/latest/UserGuide/id_roles.html
 - Elastic Load Balancing — Target Groups : <https://docs.aws.amazon.com/elasticloadbalancing/latest/application/load-balancer-target-groups.html>
 - EC2 Launch Templates : <https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/ec2-launch-templates.html>
 - AWS Auto Scaling : <https://aws.amazon.com/autoscaling/>
- Atelier AWS officiel : <https://catalog.us-east-1.prod.workshops.aws/workshops/85cd2bb2-7f79-4e96-bdee-8078e469752a/en-US>.